

מבוא למדעי המחשב

שיעור 6: אלגוריתמים

לקריאה נוספת:

- Donald E. Knuth. *The Art of Computer Programming*, volume 2, Seminumerical Algorithms. Addison-Wesley, 1998 (3rd edition).
- Donald E. Knuth. *The Art of Computer Programming*, volume 3, Sorting and Searching. Addison-Wesley, 1998 (2nd edition).

אלגוריתמים
אריتمטיים

חילוק

- נניח שאנחנו משתמשים בגרסה מנוונת של פייתון שאין בה פעולת חילוק. איך נחלק?
- שיטה א': כדי לחשב את $x // y$ נספור כמה פעמים אפשר להחסיר את y מ- x בלי לקבל תוצאה שלילית:

```
def int_div(x,y):  
    """ return the pair x//y, x%y """  
    assert(type(x) is int and type(y) is int)  
    result = 0  
    remainder = x  
    while remainder >= y:  
        result += 1  
        remainder -= y  
    return result, remainder
```

חילוק - המשיך

- שיטה ב': נחשב את התוצאה כסכום של חזקות של 2 (כלומר, נחשב את הייצוג של התוצאה בבסיס 2 משמאל לימין):

```
def fint_div(x,y):  
    """ return the pair x//y, x%y """  
    assert(type(x) is int and type(y) is int)  
    result = 0  
    remainder = x  
    while remainder >= y:  
        powerof2 = 1  
        # find the largest power of 2 <= remainder//y  
        while remainder >= 2*powerof2*y:  
            powerof2 *= 2  
        result += powerof2  
        remainder -= powerof2*y  
    return result, remainder
```

השוואת השיטות

- ב-int_div, הלולאה מתבצעת $\left\lfloor \frac{x}{y} \right\rfloor$ פעמים.
- ב-fint_div:
- הלולאה החיצונית מתבצעת לכל היותר $1 + \log_2 \left\lfloor \frac{x}{y} \right\rfloor$ פעמים (פעם אחת לכל ביט בייצוג של result בסוף).
- הלולאה הפנימית מתבצעת לכל היותר $\log_2 \left\lfloor \frac{\text{remainder}}{y} \right\rfloor$ פעמים.
- סה"כ - לכל היותר $\log_2 \left\lfloor \frac{x}{y} \right\rfloor + \log_2^2 \left\lfloor \frac{x}{y} \right\rfloor$ איטרציות.
- מה עדיף? - תשובה בהמשך ...

העלאה בחזקה

• שיטה א': נכפיל y פעמים $x \cdot x \cdot x \dots \cdot x$

```
def pow(x,y):  
    """ compute x**y """  
    assert(type(x) is float and type(y) is int)  
    result = 1.0  
    for i in range(y):  
        result *= x  
    return result
```

העלאה בחזקה (המשך)

• שיטה ב': אם y זוגי אז $x^y = (x^2)^{y/2}$, אחרת $x^y = x \cdot (x^2)^{(y-1)/2}$

```
def fpow(x,y):  
    """ compute x**y """  
    assert(type(x) is float and type(y) is int)  
    result = 1.0  
    while y > 0:  
        if y%2 == 1:  
            result *= x  
        x *= x    # for the next iteration, square x,  
        y //= 2    # and divide the power by 2.  
    return result
```


השוואת השיטות

- ב-pow, הלולאה מתבצעת y פעמים.
- ב-fpow, הלולאה מתבצעת לכל היותר $\log_2 y$ פעמים.
- מה עדיף? - תשובה בהמשך ...

ברקורסיה

Python 3.4.1: fast_power.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - פיתרון/Code/fast_power.py

```
def fast_power(x, y):  
    """  
    This function computes x to the power of y (x**y), using the binary  
    exponentiation algorithm.  
  
    input: type(x) == float and type(y) == int and y >= 0  
    output: a float number that equals x**y  
    errors and exceptions: none  
  
    performance guarantees: O(log y) time and space  
    """  
  
    if y == 0:  
        return 1.0  
    elif y%2 == 0:  
        return fast_power(x*x, y//2)  
    else:  
        return x * fast_power(x*x, y//2)
```

ברקורסיה

Python 3.4.1: fast_power.py - /Users/yrabani/Documents/5 פייטון/תשעה/שיעור/Code/fast_power.py

```
def fast_power(x, y):
    """
    This function computes x to the power of y (x**y), using the binary
    exponentiation algorithm.

    input: type(x) == float and type(y) == int and y >= 0
    output: a float number that equals x**y
    errors and exceptions: none

    performance guarantees: O(log y) time and space
    """
    print(x, y)
    if y == 0:
        return 1.0
    elif y%2 == 0:
        return fast_power(x*x, y//2)
    else:
        return x * fast_power(x*x, y//2)
```

ברקורסיה

Python 3.4.1: fast_power.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - Code/fast_power.py

```
def fast_power(x, y):  
    """  
    This function computes x to the power of y (x**y), using the binary  
    exponentiation algorithm.
```

```
    input: type(x) == float and type(y) == int and y >= 0  
    output: a float number that equals x**y  
    errors and exceptions: none
```

```
    performance guarantees: O(log y) time and space  
    """
```

```
    print(x, y)
```

```
    if y == 0:  
        return 1.0  
    elif y%2 == 0:  
        return fast_power(x*x, y//2)  
    else:  
        return x * fast_power(x*x, y//2)
```

>>> fast_power(2,10)

2 10

4 5

16 2

256 1

65536 0

1024.0

מחלק משותף גדול ביותר

- הגדרה: מחלק משותף גדול ביותר (ממג"ב או gcd) של שני מספרים טבעיים m ו- n , הוא המספר הטבעי הגדול ביותר שמחלק את שניהם.
- איך מוצאים ממג"ב? - פתרון בי"ס יסודי:
- מפרקים את שני המספרים לגורמים ראשוניים
- הממג"ב הוא מכפלת הגורמים המשותפים
- בעייה: לא ידוע אלגוריתם יעיל לפירוק מספר לגורמים ראשוניים!

אלגוריתם אוקלידס

- נרשום $m = q \cdot n + r$ (כלומר, $q = m // n$ ו- $r = m \% n$)
- אם גורם g מחלק גם את m וגם את n , אז הוא מחלק גם את r .
- לכן, במקום לחשב $\text{gcd}(m, n)$ אפשר לחשב $\text{gcd}(n, r)$
- שימו לב ש- $r < n$, אז הרווחנו מבחינת גודל המספרים
- מתי נעצור? $\text{gcd}(k, 0) = k$

מימוש

```
def gcd(m,n):  
    """ return the greatest common divisor of m, n """  
    assert(type(m) is int and type(n) is int)  
    assert(m >= 0 and n >= 0)  
    while n > 0:  
        r = m%n  
        m = n  
        n = r  
    return m
```

מימוש

```
def gcd(m,n):  
    """ return the greatest common divisor  
    assert(type(m) is int and type(n) is int)  
    assert(m >= 0 and n >= 0)  
    while n > 0:  
        r = m%n  
        m = n  
        n = r  
    return m
```

השמורה של הלולאה: בסוף כל איטרציה, התוצאה הרצויה היא $\text{gcd}(m,n)$.

הוכחה: $\text{gcd}(m,n) = \text{gcd}(n, m \% n)$

מימוש

```
def gcd(m,n):  
    """ return the greatest common divisor  
    assert(type(m) is int and type(n) is int)  
    assert(m >= 0 and n >= 0)  
    while n > 0:  
        r = m%n  
        m = n  
        n = r  
    return m
```

השמורה של הלולאה: בסוף כל איטרציה, התוצאה הרצוייה היא $\text{gcd}(m,n)$.

הוכחה: $\text{gcd}(m,n) = \text{gcd}(n, m \% n)$

טענה: האלגוריתם מסיים תוך מספר סופי של צעדים.

הוכחה: $n < m \% n$ ולכן בכל איטרציה n קטן ולכן אחרי מספר סופי של איטרציות יגיע ל-0.

ניתוח

- כמה פעמים מתבצעת לולאת ה-while? -
- אחרי האיטרציה הראשונה מובטח לנו ש- $m > n$.
- יהי Φ סף שנקבע בהמשך.
- אם $m > \Phi \cdot n$, אז $m/\Phi < n < m\%n$.
- אחרת ($m \leq \Phi \cdot n$), נקבל $m\%n \leq m - n \leq (\Phi - 1) \cdot n$.
- אנחנו רוצים שיתקיים $m/\Phi \leq n/\Phi \leq (\Phi - 1) \cdot n$, כי אז בשני המקרים המכפלה $m \cdot n$ קטנה בפקטור לפחות Φ .
- נבחר Φ כך ש- $\Phi - 1 = 1/\Phi$, או $\Phi = (1 + \sqrt{5})/2 \approx 1.618$.
- מסקנה: מספר האיטרציות הוא לכל היותר $1 + \log_{\Phi}(m \cdot n)$.

ברקורסיה

- תזכורת: אלגוריתם אוקלידס למציאת מחלק משותף גדול ביותר מחליף את החישוב של $\text{gcd}(m, n)$ בחישוב של $\text{gcd}(n, m \% n)$.
- נתרגם זאת "מילולית" לקוד:

Python 3.4.1: buggyGCD.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - Code/buggyGCD.py

```
def gcd(m, n):  
    """  
    The call gcd(m,n) returns the greatest common divisor of m and n.  
    Both m and n are non-negative integers.  
    If they are not, the outcome is unpredictable.  
    """  
  
    return gcd(n, m%n)
```

ברקורסיה

- תזכורת: אלגוריתם אוקלידס למציאת מחלק משותף גדול ביותר מחליף את החישוב של $\text{gcd}(m, n)$ בחישוב של $\text{gcd}(n, m \% n)$.
- נתרגם זאת "מילולית" לקוד:

Python 3.4.1: buggyGCD.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור - Code/buggyGCD.py

```
def gcd(m, n):  
    """  
    The call gcd(m,n) returns the common divisor of m and n.  
    Both m and n are non-negative.  
    If they are not, the output is not stable.  
    """  
  
    return gcd(n, m%n)
```



קוד מתוקן

Python 3.4.1: GCD.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5/Code/GCD.py

```
def gcd(m, n):  
    """  
    The call gcd(m,n) returns the greatest common divisor of m and n.  
    Both m and n are non-negative integers.  
    If they are not, the outcome is unpredictable.  
    """  
    # print(m, n)  
    if n == 0:  
        return m  
    else:  
        return gcd(n, m%n)
```

Ln: 7 Col: 6

קוד מתוקן

```
Python 3.4.1: GCD.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5/Code/GCD.py

def gcd(m, n):
    """
    The call gcd(m,n) returns the greatest common divisor of m and n.
    Both m and n are non-negative integers.
    If they are not, the outcome is unpredictable.
    """
    # print(m, n)
    if n == 0:
        return m
    else:
        return gcd(n, m%n)
```

Ln: 7 Col: 6

אם נמחק את ה-# משורת ההדפסה, מה תדפיס
הקריאה gcd(45,27)?

קוד מתוקן

```
Python 3.4.1: GCD.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5/Code/GCD.py

def gcd(m, n):
    """
    The call gcd(m,n) returns the greatest common divisor of m and n.
    Both m and n are non-negative integers.
    If they are not, the outcome is unpredictable.
    """
    # print(m, n)
    if n == 0:
        return m
    else:
        return gcd(n, m%n)
```

Ln: 7 Col: 6

אם נמחק את ה-# משורת ההדפסה, מה תדפיס

הקריאה gcd(45,27)?

45 27

27 18

18 9

9 0

9

קוד מתוקן

```
Python 3.4.1: GCD.py - /Users/yrabani/Documents/5 פייתון/תשעה/שיעור 5/Code/GCD.py

def gcd(m, n):
    """
    The call gcd(m,n) returns the greatest common divisor of m and n.
    Both m and n are non-negative integers.
    If they are not, the outcome is unpredictable.
    """
    # print(m, n)
    if n == 0:
        return m
    else:
        return gcd(n, m%n)
```

Ln: 7 Col: 6

אם נמחק את ה-# משורת ההדפסה, מה תדפיס

הקריאה gcd(45,27)?

45 27

27 18

18 9

9 0

9



זו ההדפסה של ערך הביטוי gcd(45,27)

הוצאת שורש

- צריך לחשב את $r \approx \sqrt{x}$ עד כדי שגיאה של ε .
(כלומר, $|r - \sqrt{x}| \leq \varepsilon$)

- האלגוריתם:

- מנחשים את התשובה r ובודקים אם $r^2 \approx x$
- אחרת, אם $r^2 > x$ בוחרים ניחוש קטן יותר, ואם $r^2 < x$ בוחרים ניחוש גדול יותר.
- נחזיק ונעדכן קטע $[low, high]$, הניחוש - אמצע הקטע.
- **שמורת** הלולאה (loop invariant): $\sqrt{x} \in [low, high]$

שיטת החצייה

Python 3.4.1: sqrt.py - /Users/ygrabani/Documents/פייתון/מבוא למדעי המחשב/הוראה/Code/sqrt.py

```
# compute the square root using bisection/binary search

x = float(input("Enter a positive number: "))
eps = float(input("Enter desired accuracy: "))
print("Computing the square root of ",x," within accuracy ",eps)

# if x > 1 then 1 < sqrt(x) < x, otherwise 1 >= sqrt(x) >= x
# so the interval to search is either [1,x] or [x,1]
if x > 1.0:
    low = 1.0
    high = x
else:
    low = x
    high = 1.0
# print(low,high)

while high-low > eps:
    mid = (high+low) / 2.0
    # print(mid)
    if mid*mid > x:
        high = mid
    else:
        low = mid
    # print(low,high)

print("The square root of ",x," is approximately ",low)
```

האלגוריתם הבבלי

או השיטה של הרון (Hero) מאלכסנדריה

- נתחיל מניחוש כלשהו r .
- התשובה הנכונה נמצאת בין r לבין x/r .
- נבחר את אמצע הקטע הזה בתור הניחוש הבא.

זה מקרה פרטי של שיטת ניוטון-רפסון (Newton-Raphson)
שמאפשרת למצוא פיתרון של המשוואה $f(x) = 0$ בתנאים
מסויימים על f .

מימוש האלגוריתם

Python 3.4.1: Babylonian.py - /Users/ygrabani/Documents/פייתון/מבוא למדעי המחשב/הוראה/Code/Babylonian.py

```
# compute the square root using the Babylonian method
import math

x = float(input("Enter a positive number: "))
eps = float(input("Enter desired accuracy: "))
print("Computing the square root of ",x," within accuracy ",eps)

r = x    # this is our initial guess
while math.fabs(r - x/r) > eps:    # the square root is between r and x/r
    r = (r + x/r) / 2.0

print("The square root of ",x," is approximately ",r)
```

Ln: 9 Col: 18

השוואת השיטות

- בשיטת החצייה, אורך הקטע $[low, high]$ קטן בפקטור 2 בכל איטרציה. האורך ההתחלתי הוא $|x - 1|$ וממשיכים כל עוד האורך $< \epsilon$. לכן, מספר האיטרציות הוא לכל היותר $\log_2(|x - 1| / \epsilon)$.

- בשיטה הבבלית, נסמן את הערך אחרי האיטרציה ה- n ב- $r_n = (1 + \epsilon_n) \cdot \sqrt{x}$. אז $\epsilon_{n+1} \leq 1 + \min\{\epsilon_n^2, \epsilon_n\} / 2$

מספר האיטרציות הוא בערך $\log x + \log \log(x/\epsilon)$

יעילות של חישובים

סימון אסימפטוטי

- תהיינה $f: \{1, 2, 3, \dots\} \rightarrow \{1, 2, 3, \dots\}$ ו- $g: \{1, 2, 3, \dots\} \rightarrow \{1, 2, 3, \dots\}$ שתי פונקציות מונוטוניות לא-יורדות על הטבעיים.
- נסמן $f = O(g)$ אם ורק אם יש קבועים $c > 0$ ו- $m \in \{1, 2, 3, \dots\}$ עבורם לכל $n \geq m$ מתקיים $f(n) \leq c \cdot g(n)$.
- דוגמה פשוטה: $g(n) = n$, $f(n) = 2n+1$. אפשר לקחת $c = 3$ ו- $m = 1$, כי לכל $n \geq 1$ מתקיים $2n+1 \leq 3n$.
- נסמן $f = \Omega(g)$ אם $g = O(f)$.
- נסמן $f = \Theta(g)$ אם $f = O(g)$ וגם $f = \Omega(g)$.

דוגמאות

$2n+1 = \Omega(n)$, כי לכל $n \geq 1$, $n \leq 2n+1$, אז ניקח $c = 1$.
ראינו כבר ש- $2n+1 = O(n)$, לכן, $2n+1 = \Theta(n)$.

$n^2 + 10^6 n = O(n^3)$, כי לכל $n \geq 1000$, $n^2 + 10^6 n \leq 2 \cdot n^3$, אז ניקח $c = 2$.
מצד שני, $n^2 + 10^6 n \neq \Omega(n^3)$.

$n^2 + 10^6 n = \Theta(n^2)$. למה?

לכל קבוע $a > 1$, $\log_a n = \Theta(\log_2 n)$.
זה נובע ישירות מהזהות $\log_a n = \log_2 n / \log_2 a$ (ניקח $c = 1/\log_2 a$).
לכן בסימון אסימפטוטי נכתוב $O(\log n)$ וכדומה, בלי לציין בסיס ספציפי.
 $2^n = O(3^n)$, אבל $3^n \neq O(2^n)$. בחזקות, בניגוד ללוגריתמים, הבסיס חשוב.

דוגמאות

$2n+1 = \Omega(n)$, כי לכל $n \geq 1$, $n \leq 2n+1$, אז ניקח $c = 1$.
ראינו כבר ש- $2n+1 = O(n)$, לכן, $2n+1 = \Theta(n)$.

$n^2 + 10^6 n = O(n^3)$, כי לכל $n \geq 1000$, $n^2 + 10^6 n \leq 2 \cdot n^3$, אז ניקח $c = 2$.
מצד שני, $n^2 + 10^6 n \neq \Omega(n^3)$.

$n^2 + 10^6 n = \Theta(n^2)$. למה?

הגדרה: $x = \log_a n$ אם $a^x = n$

לכל קבוע $a > 1$, $\log_a n = \Theta(\log_2 n)$.

זה נובע ישירות מהזהות $\log_a n = \log_2 n / \log_2 a$ (ניקח $c = 1/\log_2 a$).
לכן בסימון אסימפטוטי נכתוב $O(\log n)$ וכדומה, בלי לציין בסיס ספציפי.

$2^n = O(3^n)$, אבל $3^n \neq O(2^n)$. בחזקות, בניגוד ללוגריתמים, הבסיס חשוב.

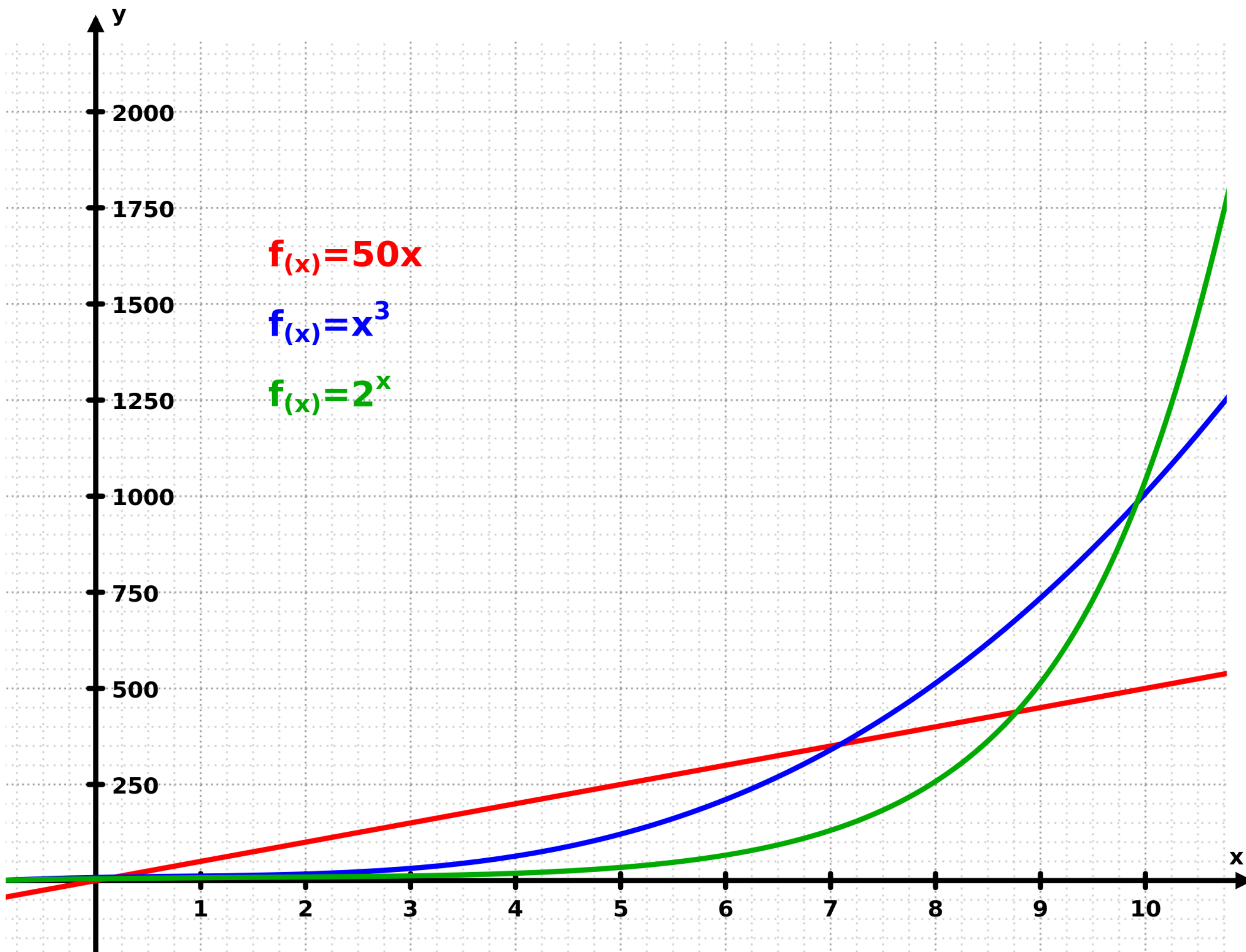
דוגמאות

$2n+1 = \Omega(n)$, כי לכל $n \geq 1$, $n \leq 2n+1$, אז ניקח $c = 1$.
ראינו כבר ש- $2n+1 = O(n)$, לכן, $2n+1 = \Theta(n)$.

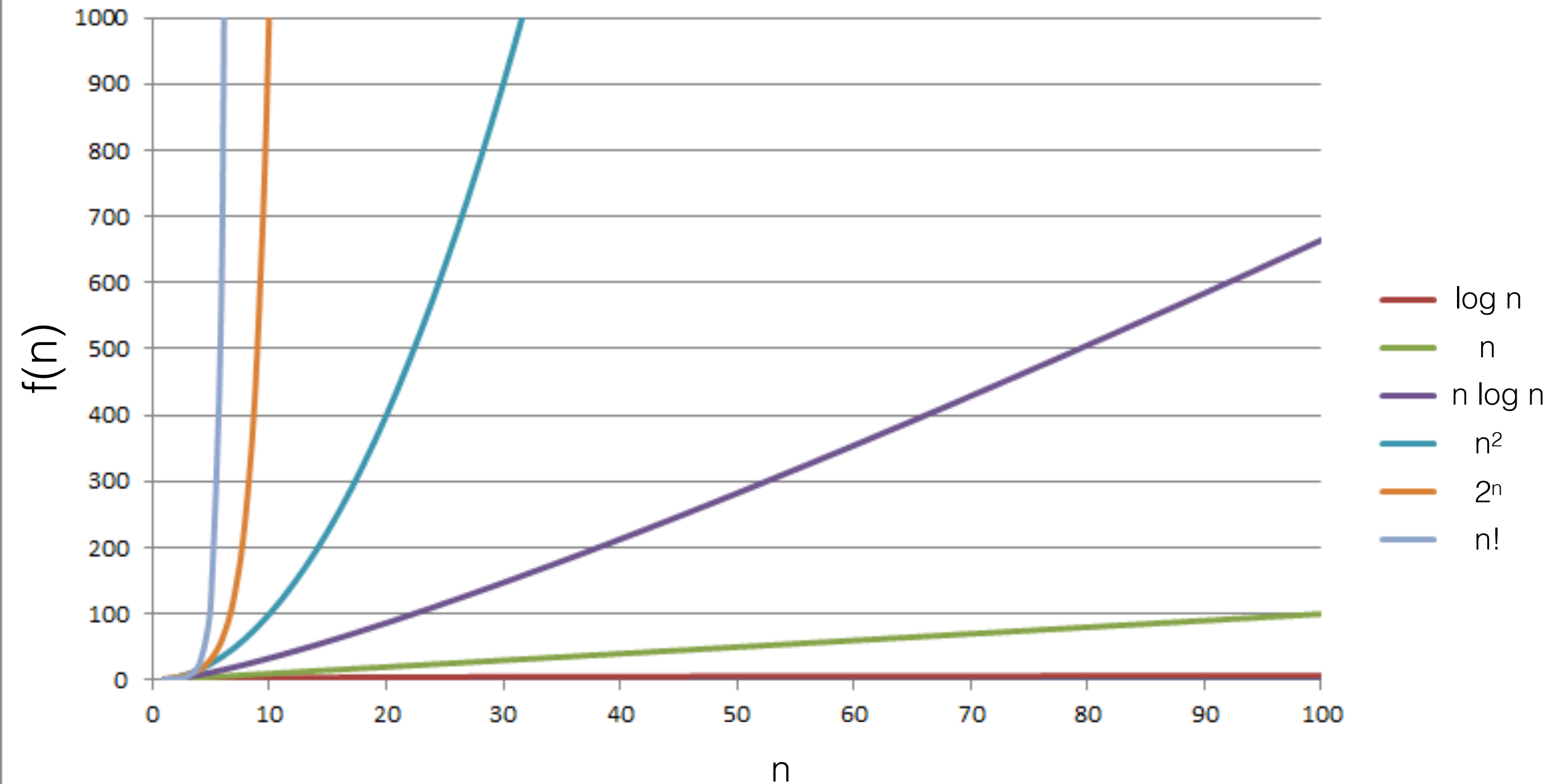
$n^2 + 10^6 n = O(n^3)$, כי לכל $n \geq 1000$, $n^2 + 10^6 n \leq 2 \cdot n^3$, אז ניקח $c = 2$.
מצד שני, $n^2 + 10^6 n \neq \Omega(n^3)$.

$n^2 + 10^6 n = \Theta(n^2)$. למה?

לכל קבוע $a > 1$, $\log_a n = \Theta(\log_2 n)$.
זה נובע ישירות מהזהות $\log_a n = \log_2 n / \log_2 a$ (ניקח $c = 1/\log_2 a$).
לכן בסימון אסימפטוטי נכתוב $O(\log n)$ וכדומה, בלי לציין בסיס ספציפי.
 $2^n = O(3^n)$, אבל $3^n \neq O(2^n)$. בחזקות, בניגוד ללוגריתמים, הבסיס חשוב.



קצב הגידול של הפונקציה



סיבוכיות

- מדד חשוב ליעילות של אלגוריתם הוא כמות המשאבים שהוא צורך כפונקציה של אורך הקלט (בביטים או בתאי זיכרון בגודל קבוע).
- המשאבים שמעניינים אותנו הם:
 - זמן - מספר הפעולות הבסיסיות הנדרשות.
 - מקום - מספר תאי הזיכרון (בנוסף לקלט) הנדרשים.
- פעולה בסיסית - כל פעולת מחשב שלוקחת זמן שלא תלוי באורך הקלט (פעולות חשבון, השמות, השוואות, ...)
- בפייתון צריך להיזהר עם הספירה - הרבה קורה "מאחורי הקלעים"
- בכל אורך קלט נתייחס למקרה הגרוע ביותר של קלט באורך זה.

סיבוכיות אסימפטוטית

- בדרך כלל תעניין אותנו ההתנהגות האסימפטוטית של סיבוכיות האלגוריתם, ולא הפונקציה המדוייקת.
- לדוגמה: למצוא מינימום של n ערכי קלט בשיטה המקובלת לוקח זמן של $\Theta(n)$.
- גישה זו מתעלמת מקבועים כפליים. למשל, סיבוכיות של n^3 יותר גרועה מסיבוכיות של $10^{100} \cdot n^2$, למרות שלכל קלט סביר הערך הראשון קטן מהשני.

השוואת אלגוריתמים

- נתונים שני אלגוריתמים A ו- B לבעייה מסויימת. רוצים להשוות, לדוגמה, את משאבי הזמן שהם דורשים.
- סיבוכיות הזמן של A היא $\Theta(f)$ ושל B היא $\Theta(g)$.
- האלגוריתמים A ו- B שווי ביצועים אם $f = \Theta(g)$.
- האלגוריתם A עדיף על B אם $f(n)/g(n) \rightarrow 0$ כאשר $n \rightarrow \infty$.
- במקרה זה נסמן $f = o(g)$ וגם $g = \omega(f)$.

למה זה טוב?

- סיבוכיות אסימפטוטית לא תלוייה בסביבת העבודה הספציפית (חומרה, מערכת הפעלה).
- סיבוכיות אסימפטוטית נותנת מושג על התנהגות האלגוריתם על קלט גדול מאוד.
- סיבוכיות אסימפטוטית נותנת מושג על התועלת הצפויה משיפור ביצועי החומרה.
- טענות לגבי סיבוכיות אסימפטוטית תקפות לכל קלט אפשרי.

למה זה רע?

- לפעמים ביצועי האלגוריתם הרבה יותר טובים על קלט אופייני (אבל קשה לאפיין מהו קלט אופייני).
- עבור קבועים מאוד גדולים (למשל 10^{100}), באופן מעשי הקבועים חשובים יותר מהאסימפטוטיקה.

מדידת זמן

```
import time

def time_f(sizes):
    for n in sizes:
        start = time.time()
        f(n)
        print("n=", n, "    time=",
              time.time() - start, "seconds")

time_f([10**5, 10**6, 10**7, 10**8, 10**9])
```

מה הסיבוכיות?

```
def f(n):  
    i=n  
    while i>0:  
        i = i-1
```

סיבוכיות ליניארית $O(n)$

```
def f(n):  
    i=n  
    while i>0:  
        i = i-1
```

```
>>>
```

```
n= 100000    time= 0.018007993698120117 seconds
```

```
n= 1000000   time= 0.08005785942077637 seconds
```

```
n= 10000000  time= 0.734501838684082 seconds
```

```
n= 100000000 time= 7.265594959259033 seconds
```

```
n= 1000000000 time= 75.43544101715088 seconds
```

```
>>>
```

מה הסיבוכיות?

```
def f(n):  
    sum = 0  
    i = 0  
    while i < n:  
        i = i + 1  
        for j in range(n):  
            sum = sum + j
```

סיבוכיות ריבועית $O(n^2)$

```
def f(n):  
    sum = 0  
    i = 0  
    while i < n:  
        i = i + 1  
        for j in range(n):  
            sum = sum + j
```

```
>>>
```

```
n= 100    time= 0.00099999275207519531 seconds
```

```
n= 1000   time= 0.06604814529418945 seconds
```

```
n= 10000  time= 7.530364990234375 seconds
```

```
>>>
```

מה הסיבוכיות?

```
def f(n):  
    while n>1:  
        n = n/2
```

סיבוכיות לוגריתמית $O(\log n)$

```
def f(n):  
    while n>1:  
        n = n/2
```

[illegible]

מה הסיבוכיות?

```
def f(n):  
    sum = 0  
    for i in range(2**n):  
        sum = sum+i
```


סיבוכיות אקספוננציאלית $O(2^n)$

```
def f(n):  
    sum = 0  
    for i in range(2**n):  
        sum = sum+i
```

```
>>>  
n= 10    time= 0.0 seconds  
n= 15    time= 0.008522987365722656 seconds  
n= 20    time= 0.09558296203613281 seconds  
n= 25    time= 2.553812026977539 seconds  
n= 30    time= 82.20163202285767 seconds  
>>>
```

דוגמאות שראינו

- חילוק $x // y$ בשיטת החזקות של 2: $O(\log^2(x/y))$ זמן
- העלאה בחזקה x^y בשיטת הריבוע: $O(\log y)$ זמן
- הוצאת שורש $x^{1/2}$ בדיוק ε :
- בשיטת החצייה: $O(\log(x/\varepsilon))$ זמן
- בשיטה הבבליית: $O(\log x + \log \log(x/\varepsilon))$ זמן
- חישוב ממג"ב $\gcd(m,n)$: $O(\log(mn))$ זמן
- חישוב מספר פיבונצ'י F_n :
- ברקורסיה: $O(\Phi^n)$ זמן, $O(n)$ מקום
- באיטרציה: $O(n)$ זמן, $O(1)$ מקום
- מגדלי האנוי עם n דיסקיות, ברקורסיה: $O(2^n)$ זמן, $O(n)$ מקום
- פילטר פשוט על תמונה $n \times m$: $O(mn)$ זמן

אלגוריתמים על מערכים

גישה לתא במערך - $O(1)$

```
def f(n):  
    a = [0] * n  
    for i in range(n):  
        a[i] = a[i-1] + i
```

```
>>>
```

```
n= 10000      time= 0.0040051937103271484 seconds
```

```
n= 100000     time= 0.029021024703979492 seconds
```

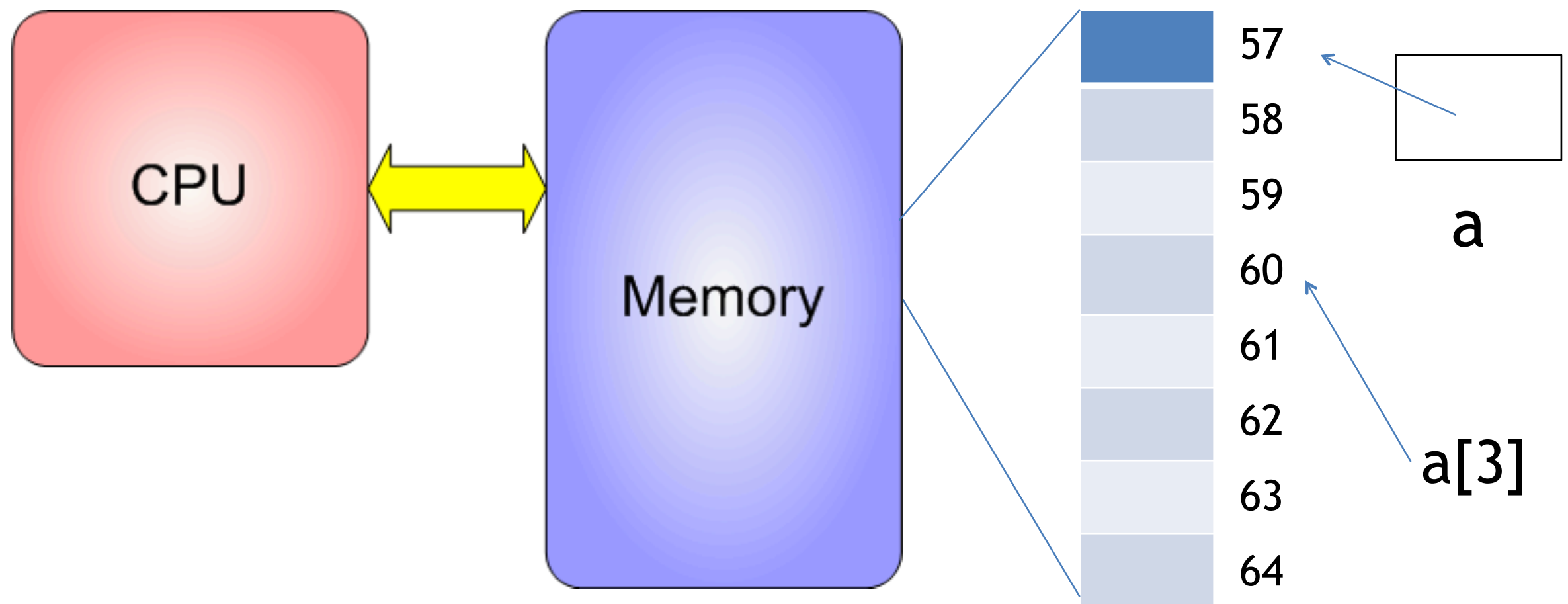
```
n= 1000000    time= 0.18813300132751465 seconds
```

```
n= 10000000   time= 1.9123399257659912 seconds
```

```
n= 100000000  time= 21.501255989074707 seconds
```

```
>>> |
```

Random Access Memory



התא $a[i]$ נמצא בכתובת $a+i$

פעולת $O(1)$ - append

```
def f(n):  
    a = []  
    for i in range(n):  
        a.append(i)
```

```
>>>  
n= 10000      time= 0.003973960876464844 seconds  
n= 100000    time= 0.021000146865844727 seconds  
n= 1000000   time= 0.12308692932128906 seconds  
n= 10000000  time= 1.2248680591583252 seconds  
n= 100000000 time= 13.74974799156189 seconds  
>>> |
```

עיקרון המימוש: אם אין מקום בהמשך הזיכרון,
מעתיקים את המערך לזיכרון בגודל כפול.

חיפוש במערך - $O(n)$

```
|  
def f(n):  
    a = [0]*n  
    for i in range(n):  
        a[i] = i  
    for i in range(n):  
        found = i in a
```

```
>>>  
n= 1000    time= 0.029036998748779297 seconds  
n= 10000   time= 0.9626638889312744 seconds  
n= 100000  time= 96.25875997543335 seconds  
>>>
```


גישה וחיפוש במילון - $O(1)$

```
def f(n):  
    d = {}  
    for i in range(n):  
        d[str(i)] = i  
    for i in range(n):  
        found = str(i) in d
```

```
>>>  
n= 100000    time= 0.14309406280517578 seconds  
n= 1000000  time= 1.0267279148101807 seconds  
n= 10000000 time= 11.48415994644165 seconds  
>>> |
```


בעיית המיון

- הקלט: מערך (list) שאיבריו ניתנים להשוואה. אנחנו נדגים עם מערך של ערכי int.
- הפלט: אותו מערך שבו סדר האיברים הוחלף כך שהם נתונים מהקטן אל הגדול.

[53, 12, 77, 23, 50] → [12, 23, 50, 53, 77]

["yossi", "david", "anat", "gila"] →
["anat", "david", "gila", "yossi"]

השוואה בפייתון

```
>>> "abc" > "aba"
```

```
True
```

```
>>> "ab" > "a"
```

```
True
```

```
>>> "A" > "a"
```

```
False
```

```
>>> "a" > "B"
```

```
True
```

```
>>> (5, 3) > (4, 4)
```

```
True
```

```
>>> 2 > "a"
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#5>", line 1, in <module>
```

```
    2 > "a"
```

```
TypeError: unorderable types: int() > str()
```

```
>>>
```

Selection Sort

```
def argmin(a, start):  
    index = start  
    for i in range(start+1, len(a)):  
        if a[i] < a[index]:  
            index = i  
    return index  
  
def sort(a):  
    for i in range(len(a)):  
        small = argmin(a, i)  
        a[i], a[small] = a[small], a[i]  
  
a = [56, 12, 34, 77, 35, 34]  
sort(a)  
print(a)
```

Selection Sort

```
def argmin(a, start):
    index = start
    for i in range(start+1, len(a)):
        if a[i] < a[index]:
            index = i
    return index

def sort(a):
    for i in range(len(a)):
        small = argmin(a, i)
        a[i], a[small] = a[small], a[i]

a = [56, 12, 34, 77, 35, 34]
sort(a)
print(a)
```

i	5	3	2	1	4
0	1	3	2	5	4
1	1	2	3	5	4
2	1	2	3	5	4
3	1	2	3	4	5
4	1	2	3	4	5

הוכחת נכונות argmin

```
def argmin(a, start):  
    index = start  
    for i in range(start+1, len(a)):  
        if a[i] < a[index]:  
            index = i  
    return index
```

משפט: argmin מחזירה את האינדקס של האיבר הקטן ביותר ב- $a[start:]$

השמורה: בסיום האיטרציה עבור i , המשתנה $index$ מחזיק את המיקום של האיבר הקטן ביותר ב- $a[start:i+1]$

הוכחת נכונות sort

```
def sort(a):  
    for i in range(len(a)):  
        small = argmin(a, i)  
        a[i], a[small] = a[small], a[i]
```

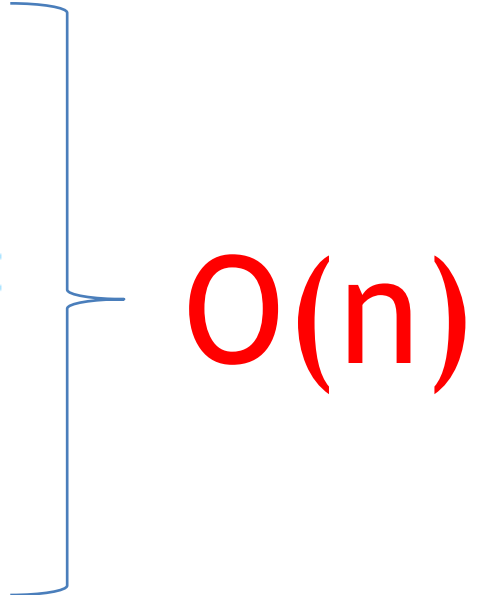
משפט: בסיום sort, המערך a מחזיק את האיברים המקוריים בסדר לא יורד.

השמורות:

1. בכל עת, a מחזיק אותה קבוצת איברים.
2. בסיום האיטרציה ה-i, $a[:i+1]$ מחזיק את $i+1$ האיברים הקטנים ביותר ב-a בסדר לא יורד.

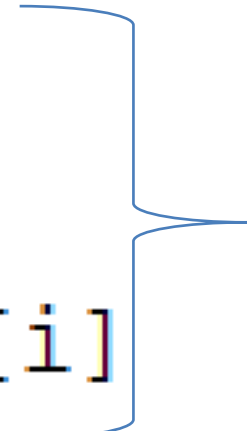
זמן ריצה - $O(n^2)$

```
def argmin(a, start):  
    index = start  
    for i in range(start+1, len(a)):  
        if a[i] < a[index]:  
            index = i  
    return index
```



$O(n)$

```
def sort(a):  
    for i in range(len(a)):  
        small = argmin(a, i)  
        a[i], a[small] = a[small], a[i]
```



$O(n)$

argmin מופעל $O(n)$ פעמים

חיפוש בינארי

```
def find(x, a):  
    l, h = 0, len(a)  
    while l < h:  
        m = (l+h)//2  
        if x == a[m]:  
            return True, m  
        if x < a[m]:  
            h = m  
        else:  
            l = m+1  
    return False, l
```


חיפוש בינארי

find(25, [10,20,30,40,50])

```
def find(x, a):  
    l, h = 0, len(a)  
    while l < h:  
        m = (l+h)//2  
        if x == a[m]:  
            return True, m  
        if x < a[m]:  
            h = m  
        else:  
            l = m+1  
    return False, l
```

l	h	m
0	5	2
0	2	1
2	2	

חיפוש בינארי

find(25, [10,20,30,40,50])

```
def find(x, a):  
    l, h = 0, len(a)  
    while l < h:  
        m = (l+h)//2  
        if x == a[m]:  
            return True, m  
        if x < a[m]:  
            h = m  
        else:  
            l = m+1  
    return False, l
```

l	h	m
0	5	2
0	2	1
2	2	

השמורה: נסמן $a[-1] = -\infty$, $a[\text{len}(a)] = \infty$
אז $a[l-1] < x < a[h]$

חיפוש בינארי

find(25, [10, 20, 30, 40, 50])

```
def find(x, a):  
    l, h = 0, len(a)  
    while l < h:  
        m = (l+h) // 2  
        if x == a[m]:  
            return m  
        if x < a[m]:  
            h = m  
        else:  
            l = m+1  
    return False, l
```

זמן ריצה: בכל איטרציה של הלולאה, אורך הקטע $[l, h]$ קטן בפקטור 2 לפחות

$O(\log \text{len}(a)) \leftarrow$ איטרציות

השמורה: נסמן $a[-1] = -\infty$, $a[\text{len}(a)] = \infty$
אז $a[l-1] < x < a[h]$

חיפוש בינארי ברקורסיה

```
def find(x, a):  
    find_in_range(x, a, 0, len(a))  
  
def find_in_range(v, a, low, high):  
    if high <= low:  
        return -1 # not found  
    mid = (high+low)//2  
    if a[mid] == x:  
        return mid  
    if a[mid]>x:  
        return find(x, a, low, mid)  
    return find(x, a, mid+1, high)
```

חיפוש בינארי ברקורסיה

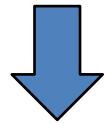
find(12, [1, 3, 5, 9, 11, 12, 15, 17])

```
def find(x, a):  
    find_in_range(x, a, 0, len(a))  
  
def find_in_range(v, a, low, high):  
    if high <= low:  
        return -1 # not found  
    mid = (high+low)//2  
    if a[mid] == x:  
        return mid  
    if a[mid] > x:  
        return find(x, a, low, mid)  
    return find(x, a, mid+1, high)
```

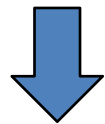
low	high	mid
0	8	4
4	8	6
4	6	5

ניתוח זמן הריצה

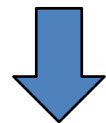
find on length n



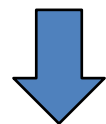
find on length $n/2$



find on length $n/4$



...



find on length 1

$O(1)$ פעולות
בכל רמה

$\log n$ רמות

מיזוג מערכים ממויינים

קלט: שני מערכים ממויינים b, c

פלט: מערך אחד ממויין a באורך $\text{len}(b)+\text{len}(c)$
שמכיל את כל האיברים מ- b ומ- c

```
def merge(b, c, a):  
    i, j, k = 0, 0, 0  
    while k < len(a):  
        if i < len(b) and (j >= len(c) or b[i] < c[j]):  
            a[k] = b[i]  
            i = i+1  
            k = k+1  
        else:  
            a[k] = c[j]  
            j = j+1  
            k = k+1
```


מיזוג מערכים ממויינים

קלט: שני מערכים ממויינים b, c

פלט: מערך אחד ממויין a באורך $\text{len}(b)+\text{len}(c)$
שמכיל את כל האיברים מ- b ומ- c

```
def merge(b, c, a):  
    i, j, k = 0, 0, 0  
    while k < len(a):  
        if b[i] < c[j]):  
            a[k] = b[i]  
            i = i + 1  
            k = k + 1  
        else:  
            a[k] = c[j]  
            j = j + 1  
            k = k + 1
```


מיזוג מערכים ממויינים

קלט: שני מערכים ממויינים b, c

פלט: מערך אחד ממויין a באורך $\text{len}(b)+\text{len}(c)$

שמכיל את כל האיברים מ- b ומ- c

זמן ריצה: $O(\text{len}(b)+\text{len}(c))$

```
def merge(b, c, a):  
    i, j, k = 0, 0, 0  
    while k < len(a):  
        if  b[i] < c[j]:  
            a[k] = b[i]  
            i = i+1  
            k = k+1  
        else:  
            a[k] = c[j]  
            j = j+1  
            k = k+1
```

מיון בעזרת מיזוג

כדי למיין רשימה, למשל $[6,2,1,3,5,4]$:

1. נחלק אותה לשתי רשימות $[6,2,1]$ ו- $[3,5,4]$

2. נמיין כל חצי (נקבל $[1,2,6]$ ו- $[3,4,5]$)

3. נמזג את שתי הרשימות הממויינות לרשימה אחת ממויינת $[1,2,3,4,5,6]$

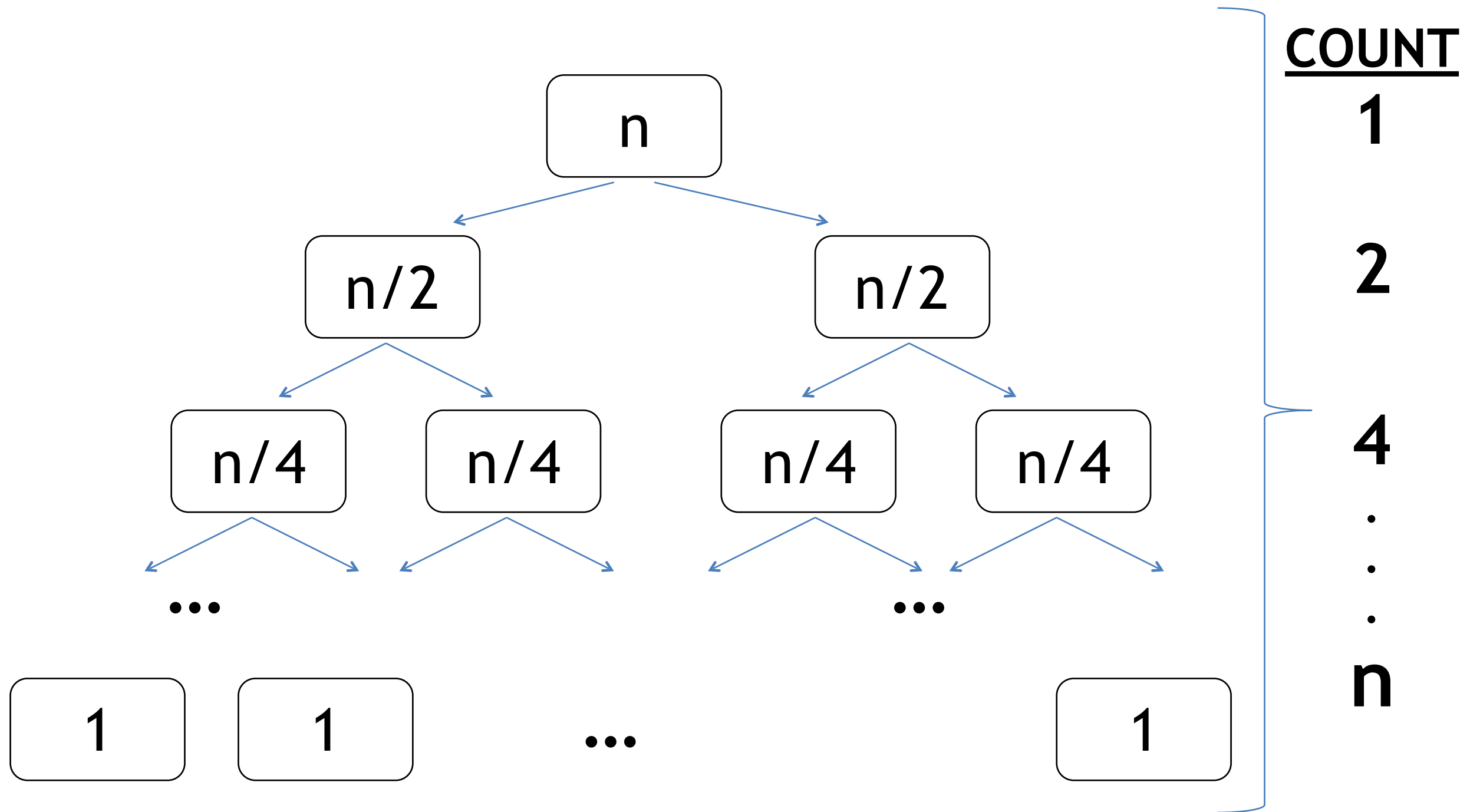
מימוש צעד 2 - על ידי הפעלת אותו מיון בעזרת מיזוג פעמיים - שימו לב שהרשימות קצרות יותר

מימוש

```
def sort(a):  
    if len(a)==1:  
        return  
    mid = len(a)//2  
    b = a[:mid]  
    c = a[mid:]  
    sort(b)  
    sort(c)  
    merge(b,c,a)
```

```
>>>  
>>> a = [5, 7, 2, 3, 8, 0]  
>>> sort(a)  
sort called on [5, 7, 2, 3, 8, 0]  
    sort called on [5, 7, 2]  
        sort called on [5]  
        sort called on [7, 2]  
            sort called on [7]  
            sort called on [2]  
            merged [7] and [2] into [2, 7]  
        merged [5] and [2, 7] into [2, 5, 7]  
    sort called on [3, 8, 0]  
        sort called on [3]  
        sort called on [8, 0]  
            sort called on [8]  
            sort called on [0]  
            merged [8] and [0] into [0, 8]  
        merged [3] and [0, 8] into [0, 3, 8]  
    merged [2, 5, 7] and [0, 3, 8] into [0, 2, 3, 5, 7, 8]  
...
```

עץ הקריאות



Run time is $O(n \log n)$

- A call to `sort(length  $k$ )` requires:
 - $O(k)$ time to split the list / copy it
 - $O(k)$ time to Merge two sub-lists of total length k
 - Recursive calls

- The total run has:
 - 1 call on length n
 - 2 calls on length $n/2$
 - ...
 - 2^i calls on length $n/2^i$
 - ...
 - n calls on length 1

$\text{Log}(n)$
levels

Each level takes
 $2^i * n / 2^i = O(n)$
time